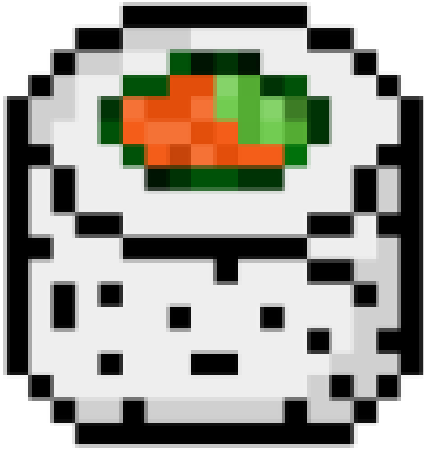




Google
Summer of Code

Urumaki Lab

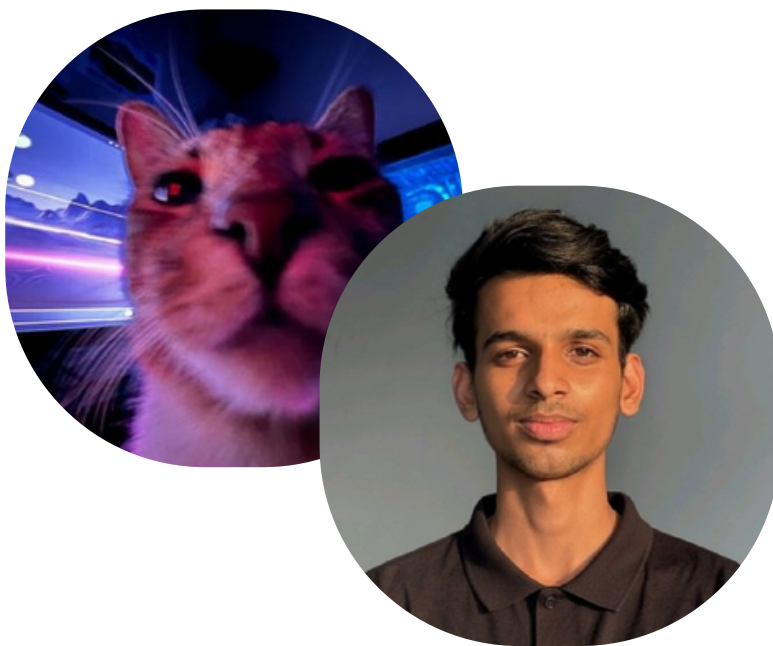
End-to-End Accessibility Evaluation Platform
&
Configuration & Disability Profile Engine

Prepared By:
Rakshit Yadav

TABLE OF CONTENTS

<u>Personal Information</u>	3
<u>Project Title</u>	4
<u>About Me</u>	4
<u>Contribution And Experience</u>	5
<u>Presence in Community</u>	7
<u>Some Personal Project</u>	7
<u>Benefits To Open Source</u>	8
<u>Project Details</u>	9
<u>Technical Overview</u>	11
<u>Deliverables</u>	29
<u>Schedule and Timeline</u>	34
<u>Availability</u>	37
<u>Conclusion</u>	37

PERSONAL INFORMATION



Name	Rakshit Yadav
Location	Rajasthan, India
Email	yadavrakshit60@gmail.com
Timezone	Indian Standard Time (IST) (UTC +05:30)
Institution	Newton School Of Technology
Program	Btech in Computer Science with Specialization in Ai-ML
Github	https://github.com/rakshityadav1868
LinKedIn	https://www.linkedin.com/in/rakshityadav07/
X	https://x.com/rakshit_yadav19

PROJECT TITLE:

End-to-End Accessibility Evaluation Platform
&
CONFIGURATION & DISABILITY PROFILE ENGINE

POSSIBLE MENTOR: AFRA PASCUAL ALMENARA

ABOUT ME

Hi, I'm Rakshit Yadav, a first-year student at Newton School of Technology in India.

In the beginning of my first semester, I got started with web development; I also delved into the fundamentals of machine learning and began contributing to some open source projects. My knowledge base now consists of an array of frontend, backend, and database technologies, and although I tend to be tech stack agnostic, I pick up whatever skills I require to finish a job that a project demands.

I started contributing to open-source projects in August of 2025 through Ruxailab, my favorite organization that I work with most. I have fixed bugs, improved CI pipelines and also contributed towards the inclusion of many major new features. So far, I've been able to manage getting 31 PRs accepted and added almost 10000 lines of code, plus over 110 commits.

I have also participated in Hacktoberfest'25, and in only the last 3 days, I was able to get a supercontributor tag, which actually motivated me to get into open-source.

[Hacktoberfest post](#)

CONTRIBUTION AND EXPERIENCE:

Issues and Pull Requests:

Issues Created: 9 created, 9 accepted and some PRs were merged without an issue being created.

AI-ACCESSIBILITY-API :

1)	Merged	Removing duplicated Docker content #2
----	--------	---

RUXAILAB:

1)	Merged	Added Email verification using custom smtp in Ruxailab #1644
2)	Merged	Added a new task option named Tam Analytics #1275
3)	Merged	Fixed all Eslint Warning in the Components directory #1635
4)	Merged	Added test for complete weight_function directory #1619
5)	Merged	Implement real time satisfaction calculator for Nasa-Tlx and Sus tasks #1228
6)	Merged	Adding Unit test for study controller #1395
7)	Merged	Moving account deletion logic from views to controllers folder. #1304
8)	Merged	Remove Unused dependencies from package.json #1204
9)	Merged	Removing Account tab from profile layout #1197
10)	Merged	Changing most used method from placeholder to real time #1128
11)	Merged	Fixed missing pre and post task in moderated test #1573
12)	Merged	Controls not showing in moderated test #1572
13)	Merged	Rendering html content properly in task desc. #1568
14)	Merged	Initialize pre and post test while pressing calibration config confirmation btn #1502

15)	Merged	Changing loading indicator while signup <u>#1456</u>
16)	Merged	Fixing loading indicator in heuristic preview and answers tab <u>#1452</u>
17)	Merged	Fixing SART answers was not stored in firestore <u>#1358</u>
18)	Merged	Added pre-text can be exported as latex language <u>#1338</u>
19)	Merged	Adding new loading indicator to main layout <u>#1320</u>
20)	Merged	Fixing logo in setting tab <u>#1293</u>
21)	Merged	Adding remove profile picture btn <u>#1248</u>
22)	Merged	Fixing answer timeline graph <u>#1214</u>
23)	Merged	Underline not visible under nasa-tlx tab in answers <u>#1184</u>
24)	Merged	Adding show more/less btn in active studies <u>#1174</u>
25)	Merged	Adding local date formatting <u>#1162</u>
26)	Merged	Adding view all btn in active studies overview <u>#1131</u>
27)	Merged	Fixing date range <u>#1119</u>
28)	Merged	Fixing study sidebar <u>#1033</u>
29)	Merged	Sidebar closing automatically <u>#1015</u>
30)	Merged	Correcting Month Name Spelling <u>#1111</u>
31)	Merged	Fixing Redirection in Moderated test <u>#1676</u>

SOME OTHER CONTRIBUTIONS:

Organization	Status	Link
Jaeger-UI	Merged	https://github.com/jaegertracing/jaeger-ui/pull/3334
Kubestellar-ui	Open	https://github.com/kubestellar/ui/pull/2421
Oaiedu	Merged	https://github.com/oaiedu/discord-quiz-bot/pull/31
Musicblock	Merged	https://github.com/sugarlabs/musicblocks/pull/4848
Musicblock	Closed	https://github.com/sugarlabs/musicblocks/pull/4856
Cal.com	Merged	https://github.com/calcom/cal.com/pull/28469

PRESENCE IN COMMUNITY:

As an open-source contributor, I was open to discussions on Discord and assisted new contributors in setting up RUXAILAB locally as well as resolving any technical issues they faced. Being very active within the community, I attend the Friday office hour.

SOME PERSONAL PROJECT:

1) PAYTM CLONE (VIRTUAL MONEY TRANSFER LOGIC)

VUEJS, MONGODB, EXPRESS, NODEJS

<https://github.com/rakshityadav1868/Paytm-clone->

Paytm Clone is a complete payment transfer application made on the basis of all core features of Paytm. The app comes with user authentication (sign-in, sign-up), account management, and P2P virtual money transfer.

2) RESUME ANALYSER:

PYTHON, FAST API, NLTK, SCIKIT-LEARN, REACT

<https://github.com/rakshityadav1868/job-resume-analyzer>

This enables an analysis of your CV with regard to the job description. It will provide your score and tell you how good or perfect your CV is against the job description.

3) NEXT PREP - INTERVIEW QUESTIONS:

PYTHON, FAST API, SPACY, HUGGING FACE, GEMINI API, REACT

<https://github.com/rakshityadav1868/NextPrep-Interview>

Next Prep examines the job description by using natural language processing (NLP) and comes up with relevant questions based on job roles, which include their level of difficulty and are also categorized. It obtains key points by using Spacy and categorizes using Hugging Face zero-shot classification, and it uses the Gemini API to generate some questions.

BENEFITS OF OPEN SOURCE:

Accessibility testing tools are mostly paid, closed-source, or limited to single-page scans. This proposal extends the GSoC 2025 repos connecting unlinked components (BLIP, AriaAligner, HierarchyGuardian) and adding multi-page crawling, WCAG-EM sampling, and a Configuration & Disability Profile Engine that makes every audit targeted, traceable, and disability-aware.

PROJECT DETAILS:

AIM:

This project covers two projects:

The goal of this project is to extend the existing GSoC 2025 accessibility repos into a complete end to end evaluation platform.

1) PROJECT 1(END-TO-END ACCESSIBILITY EVALUATION PLATFORM):

Extends both repos into a full evaluation platform multi-page crawling, WCAG-EM sampling, EN 301 549 mapping, and conformance reporting.

2) PROJECT 2(CONFIGURATION AND DISABILITY PROFILE ENGINE):

Builds the Configuration Engine, Step 0 layer defining evaluation strategy before crawling begins, including disability profiles, WCAG models, and test matrix generation.

Already Built (GSoC 2025)	What This Proposal Adds
Single page scan via (POST/api/runAccessibilityTest)	Multi page BFS crawler +WCAG-EM sampling
BLIP, anchor classifier (does not have endpoints)	BLIP + anchor classifier endpoints connected
AriaAligner, HierarchyGuardian (no routes)	Both wired up as FastAPI endpoints
Basic Firestore storage per scan	Structured collections: studies, pages, sample, findings, report
No disability tagging	Visual / Motor / Cognitive / Hearing tag on every issue
No EU standard mapping	EN 301 549 reference on every issue
No manual review queue	Human-in-loop queue with Pass / Fail / Note

No conformance report	Final WCAG-EM verdict + structured JSON export
No config validation layer	Pydantic config engine (Project 2)
No disability profile catalogue	Full Visual/Motor/Cognitive/Hearing catalogue (Project 2)
No WCAG criteria mapping	WCAG success criteria mapped to disability profiles and test types (Project 2)
No test matrix generation	Profile aware test matrix generator (Project 2)

DEPENDENCIES

1. **Pa11y + axe-core**: WCAG 2.1 and EN 301 549 accessibility scanning
2. **Puppeteer**: Chrome browser for crawling and rendering web pages
3. **BeautifulSoup + Selenium**: HTML parsing and DOM analysis for AI checks
4. **Gemini AI**: Generates fix suggestions for every accessibility issue found
5. **BLIP (HuggingFace)**: AI alt text generation for images
6. **Scikit-learn**: Anchor text classification to detect non-descriptive links
7. **Pydantic**: Config validation and schema modelling for the Configuration Engine

TOOLS AND TECHNOLOGIES:

To ensure seamless integration with RUXAILAB, the following tech stack is chosen:

1. **Vue.js**: Display accessibility audit results, manual review interface, and conformance reports inside RUXAILAB.
2. **FastAPI + Python**: Backend framework for AI analysis, disability profile mapping, and report generation
3. **Express.js + Node.js**: Backend for Pa11y and axe core scanning, page crawling and WCAG criterion mapping.
4. **Firebase**: Used to store evaluation sessions, audit finding and structured JSON datasets.

FIGMA LINK:

<https://bit.ly/4bJllaR>

PROTOTYPE:

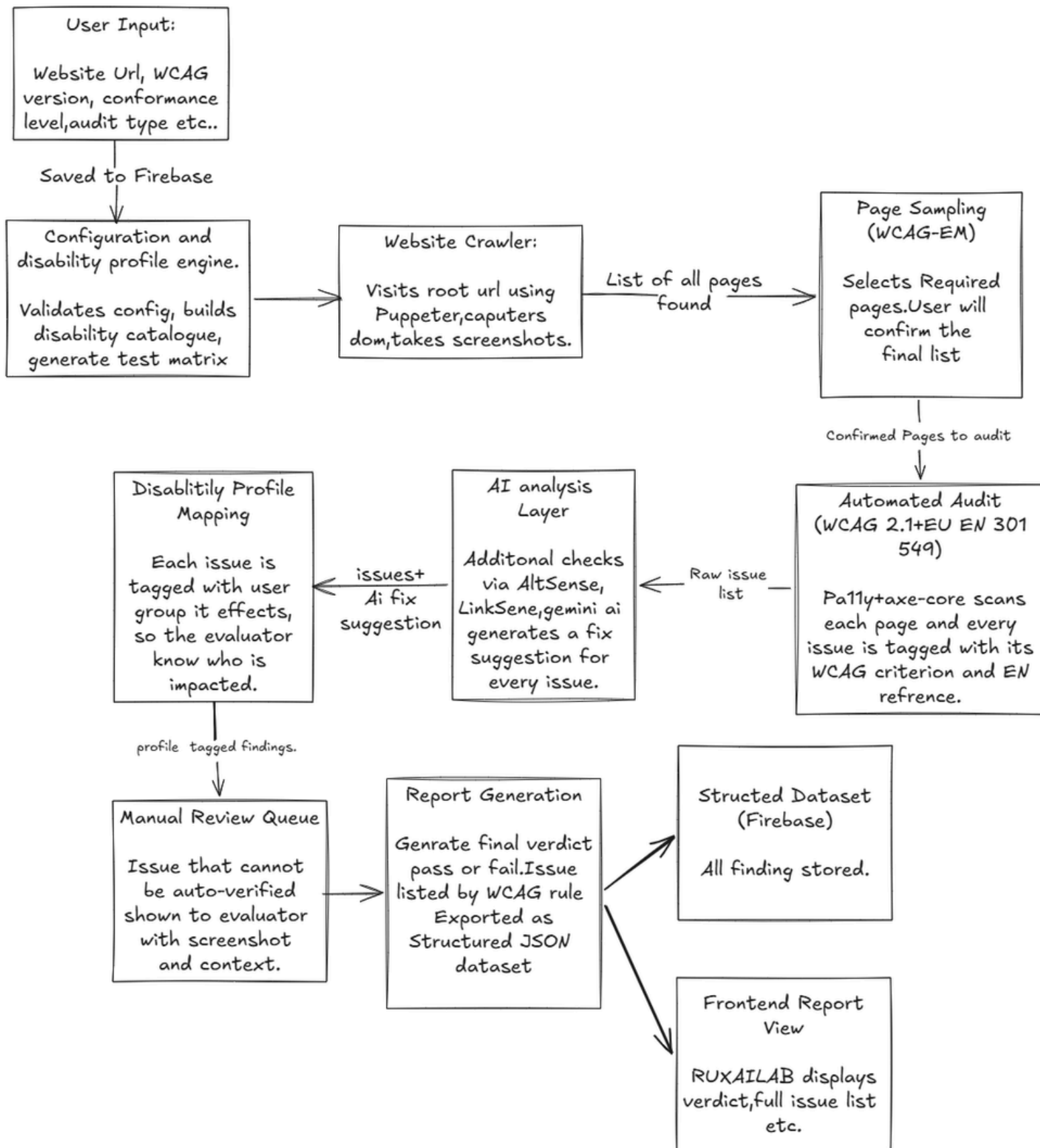
<https://www.youtube.com/watch?v=-wBgO4lTKH4>

TECHNICAL OVERVIEW:

1. USER INPUT AND STUDY CONFIGURATION
2. CONFIGURATION & DISABILITY PROFILE ENGINE (PROJECT 2)
 - A. CONFIG VALIDATION
 - B. WCAG & EN 301 549 MODELS
 - C. DISABILITY PROFILE CATALOGUE
 - D. CRITERIA MAPPING
 - E. TEST MATRIX GENERATOR
 - F. FIREBASE STORAGE
3. WEBSITE CRAWLING
4. PAGE SAMPLING (WCAG-EM)
5. AUTOMATED AUDIT (WCAG 2.1 -EU EN 301 549)
6. AI ANALYSIS LAYER
7. DISABILITY PROFILE MAPPING
8. MANUAL REVIEW QUEUE
9. REPORT GENERATION
10. FRONTEND INTEGRATION

ARCHITECTURE:

https://excalidraw.com/#json=Q1ARE1m8OnR37jugz_Mth,u4Hs12-VYGIQdI5wWu0BvQ

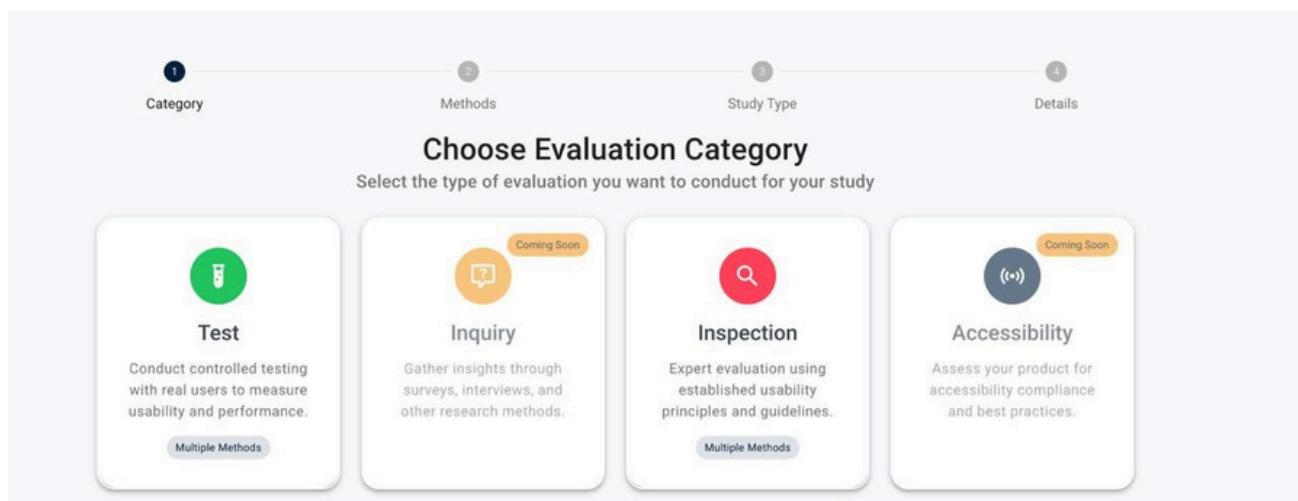


1. USER INPUT AND STUDY CONFIGURATION:

The Accessibility Evaluation workflow starts by creating a dedicated accessibility study within RUXAILAB.

• Category Selection:

The user selects the Accessibility Category in study selection.



• Audit Method Selection:

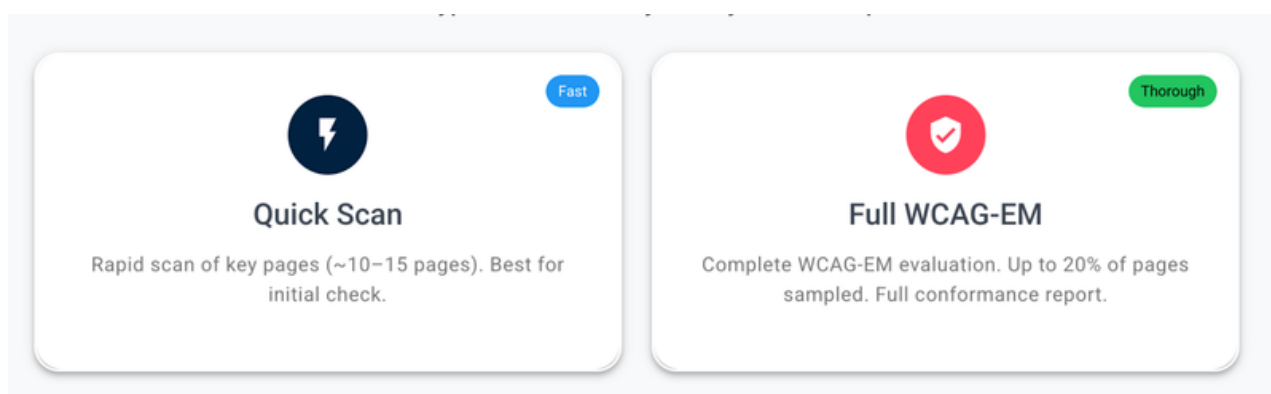
The evaluator can choose between:

a. Screening Audit:

A lightweight, rapid scan intended to identify obvious accessibility issues.

b. In-depth Audit:

Full WCAG-EM compliant evaluation including multi page sampling and structured reporting



- **Evaluation Configuration:**

The user input the parameter for the audit:

- a. Target website root URL
- b. WCAG version(2.1 or 2.2)
- c. Conformance Level (A, AA, AAA)
- d. EU EN 301 549 compliance toggle, when enabled finding are also mapped to EN 301 549 reference.
- e. Disability profiles to prioritise.

- **Study Data:**

Study Name, Description and private toggle button

Once the configuration is confirmed, the frontend will generate a unique studyId, and a structured dataset will be created in Firebase.

```
studies/{studyId}/
{
  "type": "accessibility",
  "websiteUrl": "https://example.com",
  "wcagVersion": "2.1",
  "conformanceLevel": "AA",
  "auditType": "in-depth",
  "disabilityProfiles": ["visual", "motor"],
  "euStandard": true,
  "status": "configured",
  "createdAt": timestamp
}
```

2. CONFIGURATION & DISABILITY PROFILE ENGINE :

A. Config Validation:

The engine receives the user selected config from Firebase and validates it using Pydantic models. If any field is invalid, an error is returned and crawling does not begin.

B. WCAG and EN 301 549 Models

ALL WCAG 2.1 AND 2.2 SUCCESS CRITERIA ARE DEFINED AS STRUCTURED PYTHON OBJECTS. WHEN EN 301 549 IS ENABLED, EVERY WCAG CRITERION IS AUTOMATICALLY MAPPED TO ITS CORRESPONDING EN 301 549 CLAUSE.

```
WCAG_2_1_CRITERIA = {
    "1.1.1": "Non-text Content",
    "1.4.3": "Contrast Minimum",
    "2.1.1": "Keyboard Navigation",
    "2.4.7": "Focus Visible",
    "1.2.2": "Captions",
    # 50+ criteria...
}

EN_301_549_MAP = {
    "1.1.1": "9.1.1.1",
    "1.4.3": "9.1.4.3",
    "2.1.1": "9.2.1.1",
    "2.4.7": "9.2.4.7",
    "1.2.2": "9.1.2.2",
}
```

C. Disability Profile Catalogue

Each disability profile is defined as a structured object containing affected WCAG criteria, recommended assistive technologies, and test types.

```
DISABILITY_CATALOGUE = {
    "visual": {
        "description": "Blind and low vision users",
        "criteria": ["1.1.1", "1.4.3", "1.3.4"],
        "assistive_tech": ["screen reader", "magnifier"],
    },
    "motor": {
        "description": "Keyboard only users",
        "criteria": ["2.1.1", "2.1.2", "2.4.7"],
        "assistive_tech": ["keyboard only", "switch device"],
    },
    "cognitive": {
        "description": "Reading and memory difficulty users",
        "criteria": ["3.3.1", "2.4.2", "1.3.1"],
        "assistive_tech": ["simplified UI"],
    },
    "hearing": {
        "description": "Deaf and hard of hearing users",
        "criteria": ["1.2.2", "1.2.1"],
        "assistive_tech": ["captions", "transcripts"],
    }
}
```

D. Criteria Mapping:

Every WCAG criterion is mapped to its disability profile, test type.

```
CRITERIA_MAPPING = {  
  "1.1.1": {  
    "profile": "visual",  
    "test_type": "automated",  
    "tool": "axe-core"  
  },  
  "1.4.3": {  
    "profile": "visual",  
    "test_type": "automated",  
    "tool": "axe-core"  
  },  
  "2.1.1": {  
    "profile": "motor",  
    "test_type": "manual",  
    "tool": "human reviewer"  
  },  
  "1.2.2": {  
    "profile": "hearing",  
    "test_type": "manual",  
    "tool": "human reviewer"  
  },  
}
```

E. Test Matrix

The core output of this engine. Takes validated config as input and generates an exact list of tests which criteria to check, which tool to use, and which require manual review. This matrix is directly consumed by the audit engine.

F. Firebase Storage

Generated config and test matrix are stored under studies/{studyId}/config/ in Firebase. All downstream steps crawling, audit, disability mapping, and report read from this collection.

3. WEBSITE CRAWLING :

Currently the system can only scan one page at a time. I will add a crawler that automatically discovers every page of a website starting from the root URL.

To align with website crawling, I will extend the backend with a structured crawling engine.

1. Crawling will start from the URL provided by the user.
2. Uses a confirmed test matrix from Configuration Engine to determine crawl scope and page limits.
3. Link extraction using Puppeteer rendered DOM.
4. Controlled BFS-based URL queue which ensures pages are visited in an organized way.
5. Pages from the same domain will be scanned and prevent duplicate pages from being scanned again.
6. Will keep crawl limits to avoid overloading a large website.
7. Storage of discovered pages (URL, DOM snapshot, screenshot) in Firebase and progress will be tracked and shown in the frontend

Study root document stored in Firestore:

```
{
  "type": "accessibility",
  "websiteUrl": "https://example.com",
  "status": "crawling",
  "totalPagesDiscovered": 12,
  "totalPagesScanned": 5,
  "currentUrl": "https://example.com/about",
  "progressPercentage": 40,
  "startedAt": "2026-02-28T10:00:00Z",
  "updatedAt": "2026-02-28T10:02:15Z"
}
```

Discovered Page storage:

```
{
  "url": "https://example.com/about",
  "title": "About Us",
  "depth": 1,
  "discoveredAt": "2026-02-28T10:01:00Z",
  "domSnapshot": "<html>...</html>",
  "screenshotUrl": "https://storage.googleapis.com/bucket/abc123/about.png",
  "status": "discovered"
}
```

4. PAGE SAMPLING



After the crawling phase stores all discovered pages, the sampling logic will be implemented inside the accessibility-testing-backend

This module will be aligned with WCAG-EM, which mandates us to select some pages rather than checking out an entire site.

In the accessibility-testing-backend, we intend to put in place a sampling engine that will:

1. Reads discovered pages from Firestore
2. Identifies mandatory pages
3. Groups similar pages using URL patterns and path structure
4. Applies different sampling depth based on audit type (Screening vs In-depth)
5. Mandatory page selection is also influenced by disability profiles selected in Configuration Engine
6. Stores the selected subset

The user will review and confirm the selected sample from the RUXAILAB frontend before the audit phase begins. This ensures evaluator control over the process

5. AUTOMATED ACCESSIBILITY AUDIT



Following confirmation from the sampling phase on the final list of pages, the automated accessibility audit will be carried out within the accessibility_backend. Each page will be rendered through the current use of Puppeteer and scanned for Pa11y using the axe-core engine.

Reads test matrix from Configuration Engine, only criteria present in test matrix are checked.

The audit engine will:

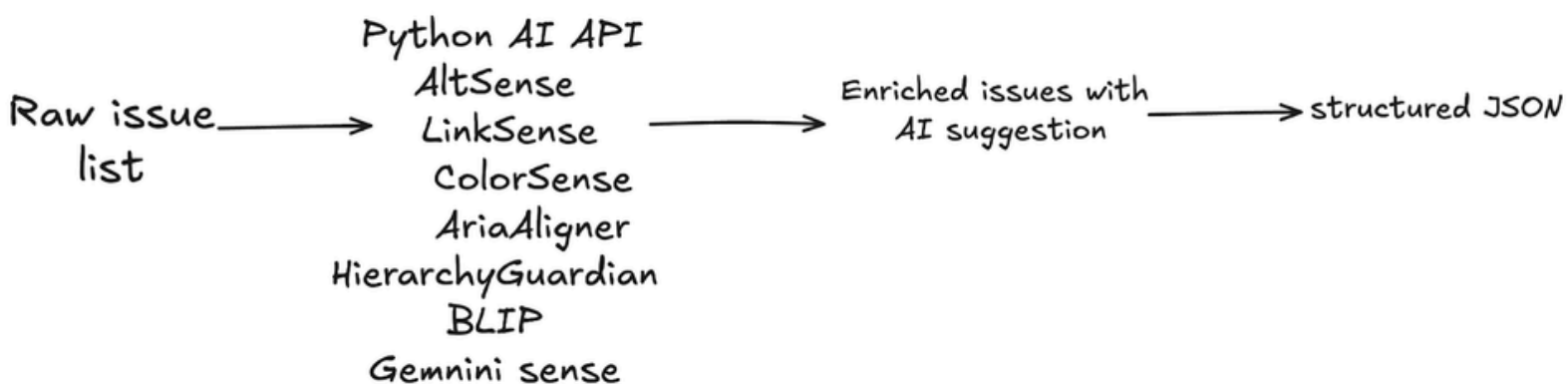
Iterate over confirmed sample pages

1. Execute Pa11y + axe-core scan per page
2. Tag each issue with:
 - WCAG Success Criterion (e.g., 1.1.1)
 - Conformance Level (A / AA / AAA)
 - EN 301 549 reference (when enabled)
 - Disability Profile tag
3. Capture Issue description, Impact level and Screenshot reference.

Will store all findings under studies/{studyId}/findings/.

WCAG tagging comes from axe-core (already integrated via Pa11y), EN 301 549 tagging will be mapped manually using a reference mapping layer.

6. AI ANALYSIS LAYER



After the audit generates the raw issue list, the result is sent to the [ai-accessibility-api](#) for intelligent post-processing and enhancement.

Currently the repo already includes Altsense, Linksense, Colorsense, and GeminiSense.

I will extend the [ai-accessibility-api](#) to:

1. Create a structured API endpoint for BLIP-based alt-text generation.
2. Will integrate the already created anchor text classification into the evaluation workflow.
3. Standardise the schema between node and Python services using Pydantic models defined in Configuration Engine
4. Attach an AI suggestion to every detected issue.
5. Return clean enriched issue objects to the node backend for Firestore.
6. Create an endpoint for the existing standalone service that currently has no routes:
 - AriaAligner
 - HierarchyGuardian

All enriched issue objects follow a standardised Pydantic schema shared between Node.js and Python services, ensuring consistent Firestore storage and uniform frontend display.

7. DISABILITY PROFILE MAPPING

Following that AI analysis layer brings enriched issues that come with fix suggestions, each one of them need to be mapped to a specific disability category it affects.

The disability profile mapping logic is defined in the Configuration Engine. This layer reads the criteria mapping from `studies/{studyId}/config/` and applies it to every enriched issue.

- 1.Receives enriched issue objects from the Python AI API
- 2.Maps each issue to disability profile using criteria mapping table from Configuration Engine
- 3.Allows the frontend to filter issues by disability profile in the report view
- 4.Attaches the profile tag to every issue object before storing in Firestore

8. MANUAL REVIEW QUEUE

Following disability profile mapping, every detail found is classified into two sections: Issues that are automatically validated and those that need human evaluation. Some WCAG criteria require a human evaluator to look at the context and decide.

I will implement a Manual Review Queue inside the Node.js backend and Vue.js frontend that:

1. Automatically identifies which issues cannot be verified programmatically based on the test matrix from Configuration Engine where test_type = "manual".
2. Places these items in a separate review queue stored in Firestore under studies/{studyId}/manualReview/
3. The RUXAILAB frontend displays each item to the evaluator with:
 - Screenshot of the element captured during the audit
 - Full HTML context of the element
 - The WCAG criterion it relates to
 - The disability profile it affects
 - AI-generated fix suggestion from Gemini
4. Evaluator marks each item as Pass, Fail, or adds a note
5. Once all items are reviewed, the queue is marked complete, and report generation begin

9. REPORT GENERATION

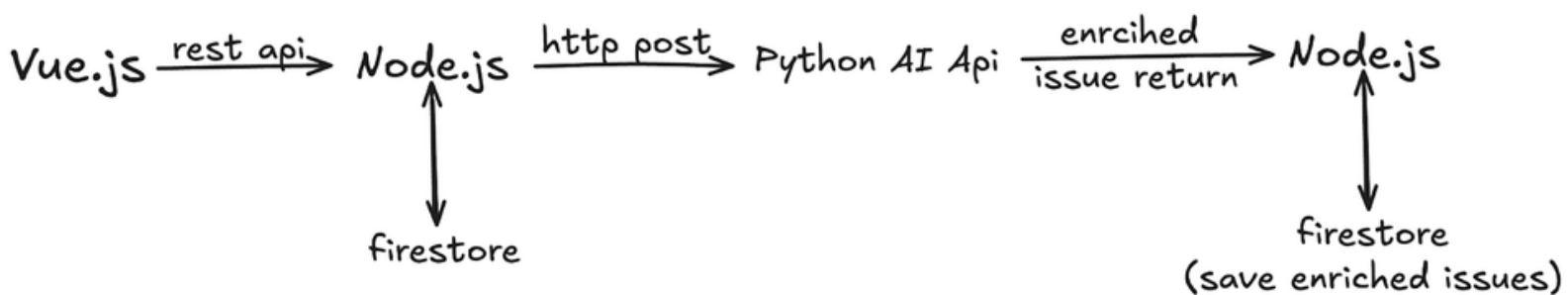
After all automated findings and manual review items are resolved, the system generates a final conformance report.

1. Collects all resolved findings from Firestore under studies/{studyId}/findings/ and studies/{studyId}/manualReview/
2. Generates final conformance verdict: Conforms or Does Not Conform to WCAG 2.1 AA and EU EN 301 549, verdict is calculated based on test matrix from Configuration Engine

3. Groups all issues by WCAG criterion, impact level, and disability profile
4. Attaches full evidence to every finding – screenshot URL, page URL, HTML selector, bounding box coordinates
5. Exports the complete report as a structured JSON dataset
6. Stores report in Firestore under studies/{studyId}/report/
7. Sends JSON to RUXAILAB Vue.js frontend for display

10. INTEGRATION

a. Integration Architecture



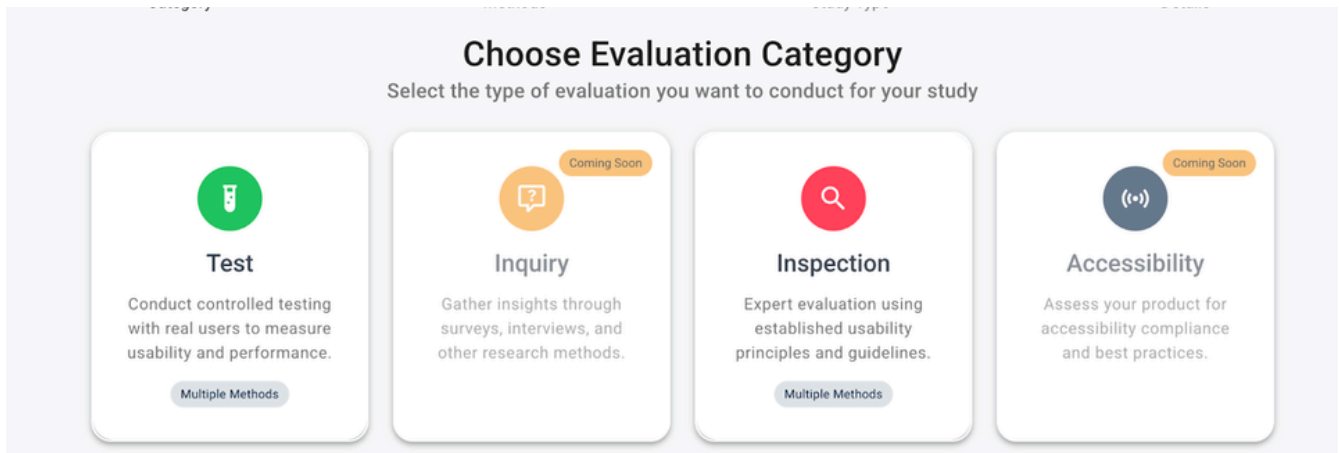
Node.js acts as a central service, managing all Firestore reads and writes. It first reads the test matrix from Configuration Engine (Step 2) stored in studies/{studyId}/config/ before crawling and auditing begins.

Once the automated scan has finished, Node.js sends raw issue lists to the Python AI API via HTTP POST requests to FastAPI endpoints. It receives issues, does AI analysis, and returns enriched data back to Node.js. The Python service does not touch Firebase directly.

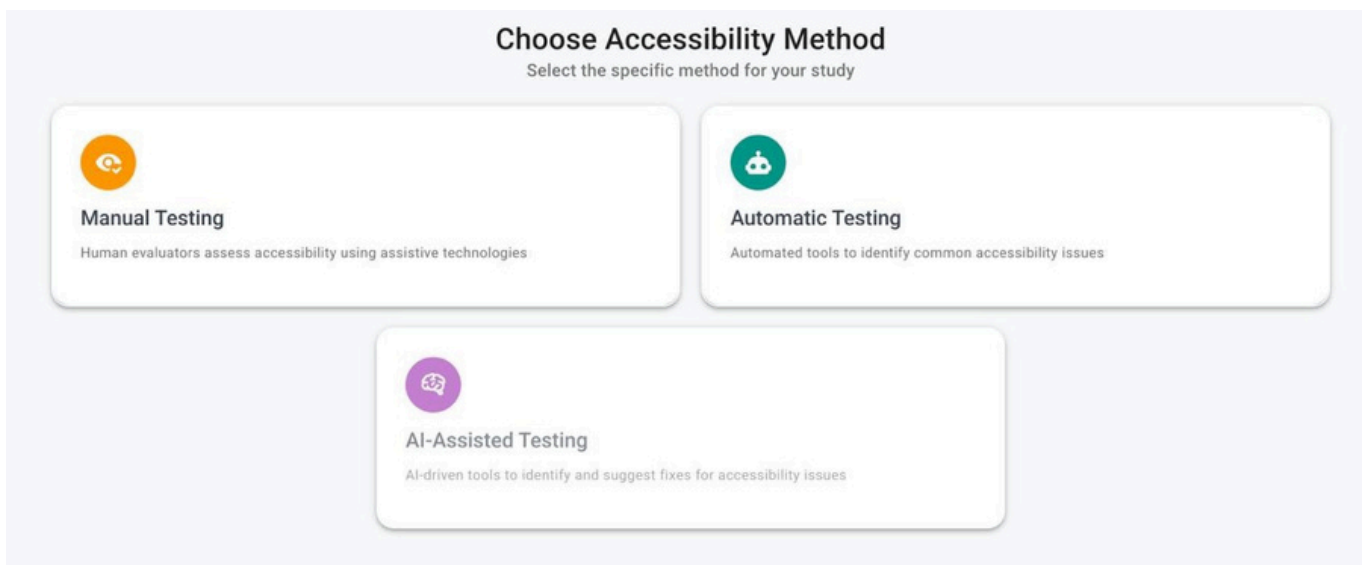
b. Frontend Architecture:

1. Study Creation:

1. Category select: select accessibility



2. **Methods:** Choose Screening Audit (quick, fewer pages) or In-depth Audit (full WCAG-EM)



3. Configuration: Select WCAG version, conformance level, EU EN 301 549 compliance, and disability profiles to prioritise in the evaluation.

WCAG Configuration

Configure the accessibility evaluation parameters.

WCAG Version

☒ WCAG 2.1 ☐ WCAG 2.2

Conformance Level

☐ A ☒ AA ☐ AAA

☐ EU EN 301 549 ⓘ

Disability Profiles

Select the disability profiles to emphasize in the evaluation.

☒ Visual ☒ Motor ☒ Cognitive ☒ Hearing

4. Details: Study name + root website URL

Study Details

Provide basic information about the accessibility evaluation.

☐ Private Study ⓘ

Configuration Summary

Audit Type
Full WCAG-EM

WCAG Version
WCAG 2.1

Level
AA

EN 301 549
Disabled

Profiles
☒ Visual ☒ Motor ☒ Cognitive ☒ Hearing

2. Manager Tab:

The Manager Tab interface for RUXAILAB displays the 'My Accessibility Audit' overview. The left sidebar contains navigation links: Manager, Evaluation, Sampling, Audit, Reports, and Settings. The main content area features a header for 'My Accessibility Audit' with an 'Accessibility Study' status and an 'Active' indicator. Below this, four summary cards show: 23 Pages Found, 0 Issues Found, 0 Pages Audited, and N/A Compliance Score. A red 'RUN EVALUATION' button is prominently displayed. The 'Evaluation Overview' section includes 'WCAG Configuration' (Version: WCAG 2.1, Conformance Level: Level AA, Profiles: Visual, Motor, Cognitive, Hearing, EU EN 301 549 ON) and 'Audit Progress' (Step 1: Scope, Step 2: Crawling, Step 3: Sampling, Step 4: Audit, Step 5: Report).

3. Evaluation Tab:

The Evaluation Tab interface for RUXAILAB displays the 'WCAG-EM 5 step evaluation methodology'. The left sidebar contains navigation links: Manager, Evaluation, Sampling, Audit, Reports, and Settings. The main content area features a 'Configuration' section with settings for WCAG Version (WCAG 2.1), Conformance Level (A, AA, AAA), Audit Type (Screening, In-depth), and EU EN 301 549 (ON). The 'WCAG-EM Progress' section shows a vertical timeline of five steps: Step 1: Define Scope, Step 2: Explore Website, Step 3: Select Sample, Step 4: Audit Sample, and Step 5: Report Findings. Below this, the 'Disability Profiles' section displays four categories: Visual (Alt text, Contrast, Zoom), Motor (Keyboard, Focus, Skip links), Cognitive (Labels, Structure, Language), and Hearing (Captions, Transcripts).

4. Sampling Tab:

RUXAILAB

Manager
Evaluation
Sampling
Audit
Reports
Settings

Page Sampling

WCAG-EM Step 3 — Select a representative sample

Pages Discovered
23

Selected for Audit
6

Awaiting Confirmation
2

Required Pages

☒	example.com	Homepage	Required
☒	example.com/login	Login	Required
☒	example.com/contact	Contact	Required
☒	example.com/nav	Navigation	Required

Random Sample

☒	example.com/about	About	Random
☒	example.com/services	Services	Random
☐	example.com/faq	FAQ	Random
☐	example.com/blog	Blog	Random

CONFIRM SAMPLE

5. Audit Tab:

RUXAILAB

Manager
Evaluation
Sampling
Audit
Reports
Settings

Automated scan results and manual review queue

Automated Checks

32 of 40 checks complete

Automated Findings

1.1.1 Missing Alt Text

Visual **Critical**

Page: example.com/home
Selector: img.hero

AI Suggestion: Add descriptive alt text that conveys the purpose and content of the image.

1.4.3 Color Contrast

Visual **Pass**

All text meets WCAG AA contrast requirements

2.1.1 Keyboard Navigation

Motor **Serious**

Page: example.com/form
Selector: button.submit

AI Suggestion: Ensure all interactive elements are keyboard accessible with visible focus indicators.

Manual Review Queue — 5 items pending

Link Purpose Context

2.4.4 Cognitive

Verify that link text provides sufficient context without surrounding text.

AI Gemini: Consider adding aria-label or more descriptive text to "click here" links to improve context.

PASS FAIL NOTE

Form Labels

3.3.2 Pending

Heading Structure

1.3.1 Pending

Video Captions

1.2.2 Pending

Focus Visible

2.4.7 Pending

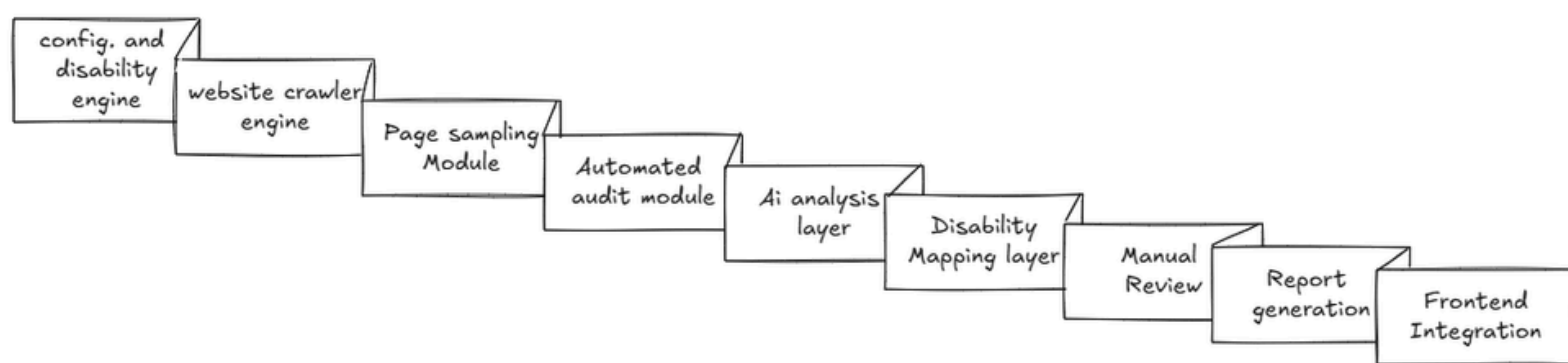
6. Report Tab

The screenshot displays the RUXAILAB Report Tab interface. On the left is a dark blue sidebar with the RUXAILAB logo and navigation links: Manager, Evaluation, Sampling, Audit, Reports (selected), and Settings. The main content area has a light gray background. At the top, a red banner with a white 'X' icon and the text 'DOES NOT CONFORM — WCAG 2.1 AA + EN 301 549' is visible. Below this, four dark blue boxes show the count of issues by severity: 8 Critical, 14 Serious, 6 Moderate, and 3 Minor. A filter bar below these boxes includes 'ALL' (selected), VISUAL, MOTOR, COGNITIVE, and HEARING. The main area lists three accessibility issues, each with a title, page URL, AI Fix suggestion, and a reference to EN 301 549:

- 1.1.1 Visual Critical**
Missing Alt Text
Page: [example.com/home](#)
AI Fix: Add descriptive alt text that conveys the purpose and content of the image.
EN 301 549: 9.1.1.1
- 2.1.1 Motor Serious**
Keyboard Navigation
Page: [example.com/form](#)
AI Fix: Ensure all interactive elements are keyboard accessible with visible focus indicators.
EN 301 549: 9.2.1.1
- 3.3.2 Cognitive Serious**
Missing Form Labels
Page: [example.com/signup](#)
AI Fix: Add clear labels to all form inputs using <label> elements or aria-label.
EN 301 549: 9.3.3.2

At the bottom, a fourth issue is partially visible: **1.4.3 Visual Moderate**. A small orange circle with a white arrow is in the bottom left corner, and a small dark blue circle with a white question mark is in the bottom right corner.

DELIVERABLES:



a) Configuration & Disability Profile Engine (Project 2):

A reusable Python configuration module that defines evaluation strategy before auditing begins.

This consists of:

- Pydantic config models for WCAG 2.1, WCAG 2.2 and EN 301 549
- Disability profile catalogue (Visual, Motor, Cognitive, Hearing) with criteria, assistive tech and test type per profile.
- WCAG criteria to disability profile mapping table
- Test type assignment (automated vs manual) for every WCAG criterion
- Profile-aware test matrix generator
- Full unit test coverage for all config models and matrix generator

b) **Website Crawler Engine:**

Our multi-page crawling system, built right into our existing Puppeteer setup in accessibility-testing-backend, is really good at finding every page of any website given just the main root URL.

This consists of:

- BFS-based URL queue and traversal mechanism
- link extraction using Puppeteer rendered DOM
- Same origin filtering and duplicate prevention
- Sitemap.xml parsing using xml2js
- Configurable crawl limits (max pages, depth, timeout)
- Storage of discovered pages (URL, DOM snapshot, screenshot) in Firebase

c) **Page Sampling Module:**

Selects a representative subset of pages for auditing instead of scanning the entire website.

This consists of:

- Required page identification (home, login, contact, navigation)
- URL pattern grouping using RegEx
- Screening vs In depth sampling depth support
- User confirmation interface in the frontend
- Storage of confirmed sample in Firebase under studies/{studyId}/sample/

d) **Automated Audit Module:**

Extended Pa1ly and axe core scanning pipeline

This consists of:

- Multi page batch scanning on the confirmed sample
- WCAG criterion mapping (1.1.1, 1.4.3, 2.1.1 and 60+ more)
- EU EN 301 549 reference tagging on every issue
- HTML snapshot per page
- Raw issue list passed to AI Analysis Layer

e) **AI Analysis Layer:**

Extended ai-accessibility-api that enriches every raw issue with AI powered checks and fix suggestions.

This consists of:

- AltSense, LinkSense, ColorSense
- BLIP AI-powered alt text generation for images (new endpoint)
- AriaAligner endpoint ARIA label checks (previously unconnected)
- HierarchyGuardian endpoint heading structure validation (previously unconnected)
- GeminiSense AI fix suggestion for every issue
- Bug fixes in existing codebase (ARIA tag fetcher, module-level test code)

f) **Disability Profile Mapping System:**

Layer that tags every accessibility issue with the disability group it affects

This consists of:

- WCAG criterion to disability profile mapping (Visual, Motor, Cognitive, Hearing)
- Profile tag attached to every enriched issue object
- Standardized Pydantic schema between Python and Node.js services

g) **Manual Review Queue:**

Human in the loop verification system for problems which can't be verified by machine automatically.

This consists of:

- Automatic identification of non auto verifiable issues
- Queue storage in Firebase under studies/{studyId}/manualReview/
- Screenshot and HTML context display per item
- Gemini AI fix suggestion per item
- Pass / Fail / Note verdict submission

h) **Report Generation Module:**

Generates the final WCAG-EM conformance report from all resolved findings.

This consists of:

- Final verdict calculation: Conforms, Does Not Conform, or Cannot Determine
- Issue grouping by WCAG criterion, impact level, and disability profile
- EN 301 549 reference attached to every finding
- Full evidence per issue screenshot, page URL
- Structured JSON dataset export
- Storage in Firebase under studies/{studyId}/report/

i) **Frontend Integration Vue.js Dashboard:**

Complete Accessibility Study flow built inside RUXAILAB following the existing design system

SCHEDULE AND TIMELINE

Community Bonding Phase	May 1 - May 24	• Interact with my mentors and finalize Project workflow • Fix existing known bugs ARIA tag fetcher, module-level test code, Dockerfile • Deep dive into accessibility-testing-backend and ai-accessibility-api codebase • Design and approve Firebase Firestore schema for studies, pages, sample, findings, and report collections. • Study Configuration Engine architecture finalize with mentor • Research WCAG criteria identify automated vs manual test types for all 50+ criteria • Design disability profile catalogue structure
Phase 1 Configuration and disability engine	May 25- June 7	• Pydantic config models for WCAG 2.1, 2.2 and EN 301 549 • Disability profile catalogue Visual, Motor, Cognitive, Hearing • WCAG criteria mapping table profile + test type + tool • Profile-aware test matrix generator • Firebase storage under studies/{studyId}/config/

Phase 4 Disability Mapping, Manual Review and Report Generation	July 6 - July 31	• Implement disability profile mapping layer, Visual, Motor, Cognitive, Hearing tagging per WCAG criterion • Build manual review queue • implement a report generation module, issue grouping by criterion and profile. • conformance verdict calculation, EN 301 549 references • Disability profile mapping uses criteria mapping table • structured JSON export, Firebase storage under studies/{studyId}/report/.
Phase 5 Frontend Integration	Aug 1 - Aug 16	• Build complete Accessibility Study flow in Vue.js. • 6 tab dashboard, Manager, Evaluation, Sampling, Audit, Reports, Settings • Integration Architecture connect Vue.js, Node.js, and Python AI API.
Final Phase Testing and Documentation	Aug 17 - Aug 24	• Write complete documentation for all three repos. • Unit testing for crawler, sampling, audit, and report modules • End to end test, full audit from URL to final report. • Final refactoring and code cleanup

AVAILABILITY:

My university examination is due to take place between the 18th of May and the 29th of May. Following this, my extended holiday break commences, and apart from GSOC '26, I do not have any obligations that would keep me busy. I will be working full-time for GSOC and therefore intend to dedicate 30 to 50 hours each week to GSOC activities. I will be most active on weekdays from Monday to Friday at 11 a. m.- 11 p. m. India Standard Time (IST), or 5:30 a. m. to 5:30 p. m. UTC.

CONCLUSION:

Thank you for reading my proposal. My document has given an elaborate explanation about both my project as well as my strategies on its execution. It aims to build a complete, Configuration & Disability Profile Engine that defines evaluation strategy, to the full End-to-End Evaluation Platform that executes it and integrates seamlessly with RUXAILAB's existing architecture.

I believe I will be able to put up with a very organised solution that can be maintained over time within the set deadline. I am very eager to be part of RuxAilab and learn a lot from the opportunity.

I AM 100% DEDICATED TO RUXAILAB AND HAVE NO OTHER PLANS FOR ANY OTHER ORGANISATIONS

I WILL BE GLAD TO BE ON BOARD. THANK YOU.